

Object Interaction Modeling

Student Study Edition · Golden Master design · 14 structured lessons with original course visuals, examples, activities, and study aids.

01 · Object Interaction Modeling

02 · Where OIM Fits in the Methodology

03 · Foreground vs Background Processing

04 · The OIM Process & Nontrivial Steps

05 · Scenarios & Scenario Tables

06 · Library Checkout: From Request to Object Story

TOPIC 08 · STUDY MAP

Contents

Use this map to move through the topic in the same lesson order as The Educator course pages.

01	Object Interaction Modeling	3
02	Where OIM Fits in the Methodology	6
03	Foreground vs Background Processing	7
04	The OIM Process & Nontrivial Steps	8
05	Scenarios & Scenario Tables	10
06	Library Checkout: From Request to Object Story	12
07	Scenario Table → UML Sequence Diagram	14
08	Analysis vs Design Sequence Diagrams	18
09	Messages, Returns & Flow of Control	19
10	Loops, Guards, Creation & Object Instances	21
11	From Sequence Diagram to Implementation	23
12	Worked Examples: Read Real UML Sequence Diagrams	27
13	Practice Studio	30
14	Common Mistakes, Agile Guidance & Exam Checklist	32

TOPIC 08 · LESSON 01 · 1 OF 14 LESSONS

TOPIC 08 · CHAPTER 9

Object Interaction Modeling

Turn a use-case request into a precise story of how software objects collaborate behind the interface.

LEARNING DESTINATION

Identify background processing, write scenarios and scenario tables, translate them into sequence diagrams, distinguish analysis from design, and connect messages to responsibilities and implementation.

PRACTICE LAB · TOPIC 08**Mission 08 — Make the Objects Tell the Story**

Model WaselNow across 14 stages: identify nontrivial background work, build scenarios, choose collaborating objects, order messages, inspect flawed sequence diagrams, reason about controllers, guards, loops and creation, and connect messages to methods.

THE BIG IDEA

What happens after the actor clicks?



Actor-system interaction models the visible foreground. **Object Interaction Modeling (OIM)** specifies how objects interact to perform the background processing of a business process.

LEARNING PATH

One transformation, step by step



1 · Find the work

Locate nontrivial use-case steps that require processing beyond presentation.



2 · Tell the scenario

Describe subject, action, object acted upon, and required data.



3 · Structure it

Organize the interaction in a scenario table.



4 · Visualize time

Translate the scenario into a sequence diagram.



5 · Design responsibilities

Use messages to expose responsibilities and methods.



6 · Review semantics

Check ordering, nesting, calls, guards, loops and creation.

RUNNING EXAMPLE

Library Checkout will grow with you

Instead of isolated fragments, this topic repeatedly returns to the lecture's Library Checkout case: Patron submits call numbers → GUI delegates → controller coordinates → DBMgr retrieves/saves → Loan is created → Document availability changes → result returns. Each later lesson adds one modeling layer.

MENTAL MODEL

USE-CASE STEP → SCENARIO → SCENARIO TABLE → SEQUENCE DIAGRAM → RESPONSIBILITY → METHOD.

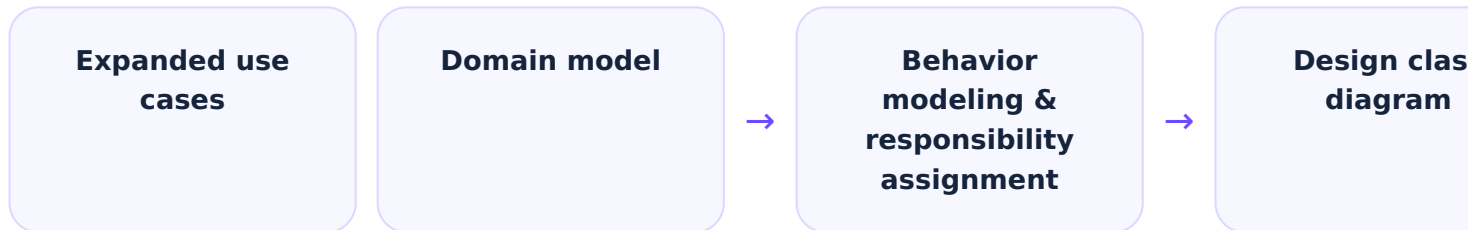
TOPIC 08 · LESSON 02 · 2 OF 14 LESSONS

TOPIC 08 · LESSON 02

Where OIM Fits in the Methodology

OIM is not an isolated UML exercise. It sits between understanding a use case and assigning software responsibilities.

Inputs → OIM → downstream design



During an iteration, expanded use cases tell us what behavior must occur; the domain model tells us which concepts exist. OIM explores how collaborating objects can realize that behavior. Its result contributes to behavior models and later design-class responsibilities.

Analysis question vs design question

Requirements / analysis

What does the actor need the system to accomplish?

Object interaction

Which objects collaborate, in what order, to produce that result?

Why this matters

Jumping directly from a use case to code hides responsibility decisions. OIM makes those decisions inspectable before implementation.

TOPIC 08 · LESSON 03 · 3 OF 14 LESSONS

LESSON 03

Foreground vs Background Processing

The screen is only the visible edge of the use case.



Actor-System Interaction

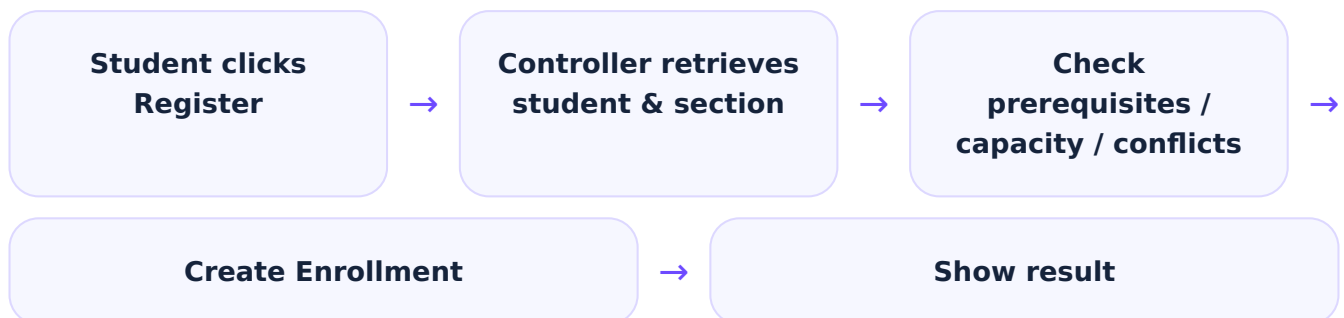
Foreground processing: acquiring actor input/action and displaying system responses.



Object Interaction

Background processing: objects collaborate, high-level algorithms process requests, and system responses are produced.

Example: course registration



The first and last boxes are visible foreground behavior. The middle collaboration is the background behavior OIM is designed to expose.

DIAGNOSTIC QUESTION

If the GUI disappeared, what processing would still have to happen to satisfy the request? That is the interaction you are trying to model.

TOPIC 08 · LESSON 04 · 4 OF 14 LESSONS

LESSON 04

The OIM Process & Nontrivial Steps

Model selectively: first find the use-case steps that actually require background processing.

The five lecture steps

1 · Collect

Collect information about business processes.

2 · Describe

Describe scenarios for nontrivial steps.

3 · Structure

Convert a scenario to a scenario table if useful.

4 · Visualize

Convert the scenario/table to a sequence diagram when the visualization adds value; the lecture treats the OIM sequence diagram as optional.

5 · Inspect

Check the OIM model for desired properties.

How do I identify a nontrivial step?

Signal	Classification	Reason
System simply displays a menu/ input dialog	Trivial	No background processing.
Same fixed response for all actors	Usually trivial	Presentation dominates.

Signal	Classification	Reason
Response differs by actor/data/condition	Nontrivial	Processing is needed to determine the response.
Objects must retrieve/check/create/update/save	Nontrivial	Clearly goes beyond GUI behavior.

Mark it in the expanded use case

The lecture recommends identifying and marking nontrivial steps in the right column, then beginning the scenario with the step on the left that triggers that processing.

TOPIC 08 · LESSON 05 · 5 OF 14 LESSONS

LESSON 05

Scenarios & Scenario Tables

Before drawing arrows, tell a disciplined object-interaction story.

An interaction sentence has four parts

Subject

Who performs the action?

Action

What does the subject do?

Object acted upon

Who/what receives the action?

Other data / objects

What information is carried?

Five-column scenario table

Step	Subject	Action	Object acted upon	Other data/ objects
4.1	Checkout GUI	checks out documents with	Checkout Controller	call numbers
4.3.1	Checkout Controller	gets document from	DBMgr	document call number
4.3.3.2.1	Checkout Controller	creates	Loan	patron p, document d
4.4	Checkout Controller	returns	Checkout GUI	msg

Construction guidelines

- Specify the **normal scenario first**: assume things go as expected.
- Add alternative flows only when needed.
- Keep the model simple; leave coding detail to implementation.
- Leave exception-handling detail to coding.
- A prototype can be useful when the design idea needs validation.

WHY THE TABLE MATTERS

It is not bureaucracy. Its columns map naturally to sequence-diagram participants and messages.

TOPIC 08 · LESSON 06 · 6 OF 14 LESSONS

LESSON 06 · LECTURE RUNNING EXAMPLE

Library Checkout: From Request to Object Story

This is the backbone example for the rest of Topic 08.

Trigger

Expanded use-case step

The patron enters the call numbers of documents to be checked out and clicks Submit.

Normal interaction scenario

4.1

Checkout GUI asks Checkout Controller to check out the documents using their call numbers.

4.2

Checkout Controller creates a blank result message.

4.3

For each call number, the controller asks DBMgr for the Document.

4.3.3

If the document exists, the controller checks whether it is available.

4.3.3.2

If available, create Loan(p,d), set Document unavailable, save Loan and Document, and record success.

4.4

Checkout Controller returns msg to Checkout GUI.

4.5

Checkout GUI displays msg to Patron.

Alternative outcomes already visible

Document available

Create Loan, change availability, persist objects.

Document unavailable

Write “document not available” to msg.

Document not found

Write “document not found” to msg.

DO NOT DRAW YET

At this stage, make sure the object story is sensible. The next lesson translates this same story into UML sequence notation.

TOPIC 08 · LESSON 07 · 7 OF 14 LESSONS

LESSON 07

Scenario Table → UML Sequence Diagram

Translate a scenario into UML interaction notation. Time runs downward; messages show who communicates with whom and in what order.

UML Sequence Diagram notation — source visual

The reference defines a sequence diagram as an interaction diagram focused on message exchange between lifelines. The figure below is the actual notation overview published by [uml-diagrams.org](https://www.uml-diagrams.org).

External source visual referenced by the course page

Major elements of a UML Sequence Diagram. Original source: [uml-diagrams.org](https://www.uml-diagrams.org) — UML Sequence Diagrams ↗ .

Source: <https://www.uml-diagrams.org/sequence-diagrams.html>

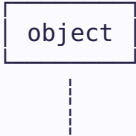
Study priority: master the Core notation first. Advanced/reference symbols are useful for reading richer diagrams, but they should not distract you from the central skill: translating a scenario into correctly ordered interactions.

DECODING KEY

Sequence Diagram Notions, Notations & Meanings

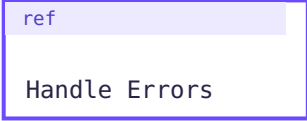
CORE — expected working vocabulary

Notion	Notation	Meaning / Semantics	Quick example
Actor		An external role that participates in the interaction but is outside the software being modeled.	<code>Patron</code> starts checkout.
Object / Participant	<code>object:Class</code>	A participating instance shown at the top of a lifeline.	<code>checkout:CheckoutGUI</code> I

Notion	Notation	Meaning / Semantics	Quick example
Lifeline		The vertical timeline of a participant. Lower events occur later.	Follow <code>:DBMgr</code> downward.
Execution / Activation		A period during which a participant is executing behavior or has control.	The controller executes <code>process()</code> .
Synchronous Message		A call where the sender normally waits for completion.	<code>getDocument(cn)</code>
Asynchronous Message		A message that does not require the sender to wait.	Send a notification event.
Return Message		An optional dashed reply showing a returned result or control.	<code>Document d</code>
Guard		A Boolean condition controlling whether behavior executes.	<code>[available]</code>
Combined Fragment		A framed region expressing repetition, alternatives, or optional behavior.	<code>loop [for each item]</code>
Object Creation		Creates a new participant; its lifeline begins at creation.	<code>«create» Loan</code>

ADVANCED / REFERENCE — recognize when encountered

Notion	Notation	Meaning / Semantics	Quick example
Object Destruction		The X marks the end of a participant's existence.	Destroy a temporary session.

Notion	Notation	Meaning / Semantics	Quick example
Interaction Use		A ref frame references another interaction instead of redrawing it.	ref Checkout
Gate		A connection point allowing a message to enter or leave an interaction boundary.	External message enters a frame.
Duration Constraint		A timing constraint on the allowed duration between occurrences.	Response within 200 ms.

Reading rule: participants are arranged horizontally, **time moves downward**, arrows show communication, activation bars show execution, and frames/guards explain control logic.

The four translation cases

1 · Actor → Object

Actor sends input/action to the object acted upon.

2 · Object → Actor

Object communicates a message/result to the actor.

3 · Object → Object

One software object invokes another.

4 · Self-call

Subject and object acted upon are the same object.

Read a simple interaction

Patron**:CheckoutGUI****:CheckoutController****:DBMgr**

1. Patron → CheckoutGUI: uid, call# list
2. CheckoutGUI → Controller: process(uid, callNumList)
3. Controller → DBMgr: getDocument(cn)
4. DBMgr → Controller: Document d
5. Controller → CheckoutGUI: msg
6. CheckoutGUI → Patron: msg

HOW TO READ ANY SEQUENCE DIAGRAM

Start at the top, identify every lifeline, then move downward one message at a time: sender → message → receiver → resulting execution or reply. Never read it left-to-right as if it were a flowchart.

TOPIC 08 · LESSON 08 · 8 OF 14 LESSONS

LESSON 08

Analysis vs Design Sequence Diagrams

Similar notation can answer two very different questions.

Analysis

- Application problem-oriented
- Models the application domain
- Describes what the world is
- Project-oriented decisions
- Should allow design alternatives

Design

- Software solution-oriented
- Models the software system
- Prescribes a software solution
- System-oriented decisions
- Usually reduces implementation choices

Lecture contrast: library system

Manual analysis:
Librarian receives ID
card



Librarian finds
patron folder



Software design: GUI
→ Controller →
DBMgr

An analysis sequence diagram can model an existing manual operation with little design decision. A design sequence diagram introduces software objects and assigns solution responsibilities.

Exam trap

Do not call a diagram “design” merely because it uses UML. Ask whether its participants represent the problem world or a chosen software solution.

TOPIC 08 · LESSON 09 · 9 OF 14 LESSONS

LESSON 09

Messages, Returns & Flow of Control

A design message can become a precise function call—not just an informal arrow label.

Read the message signature

```
msg := verify(uid:String, password>Password) : String
```

verify

Function/operation being called.

uid, password

Parameters.

String / Password

Parameter types.

msg : String

Return value and return type.

Correct causal order

- 1 · Patron → LoginGUI: uid, password
- 2 · LoginGUI → LoginController: verify(uid,password)
- 3 · LoginController → LoginGUI: msg
- 4 · LoginGUI → Patron: msg

FLOW-OF-CONTROL RULE

The triggering event must occur before processing. Nested calls happen during the caller's execution; separate calls require separate execution intervals.

TOPIC 08 · LESSON 10 · 10 OF 14 LESSONS

LESSON 10

Loops, Guards, Creation & Object Instances

Add control semantics only after the basic interaction order is correct.

**Loop**

Repeat an interaction, e.g. for each call number.

**[guard]**

Execute only when a condition is true, e.g. [d!=null].

**create**

Create a new object, e.g. l:Loan.

**Return**

Communicate a value back to the caller when useful.

Library Checkout control story

```
loop [for each cn in callNumList]
```

```
  d := DBMgr.getDocument(cn)
```

```
[d != null] a := d.isAvailable()
[a] create Loan(u,d)
[a] d.setAvailable(false)
[a] DBMgr.saveLoan(l); saveDocument(d)
```

Representing object instances

Notation	Meaning
myCar:Car	Named instance with a type; commonly used.
:Car	Unnamed instance with a known type.
myCar:	Named instance where type is unimportant/unknown/to be determined.
* :Car	Multiple instances or a collection of objects.

TOPIC 08 · LESSON 11 · 11 OF 14 LESSONS

LESSON 11

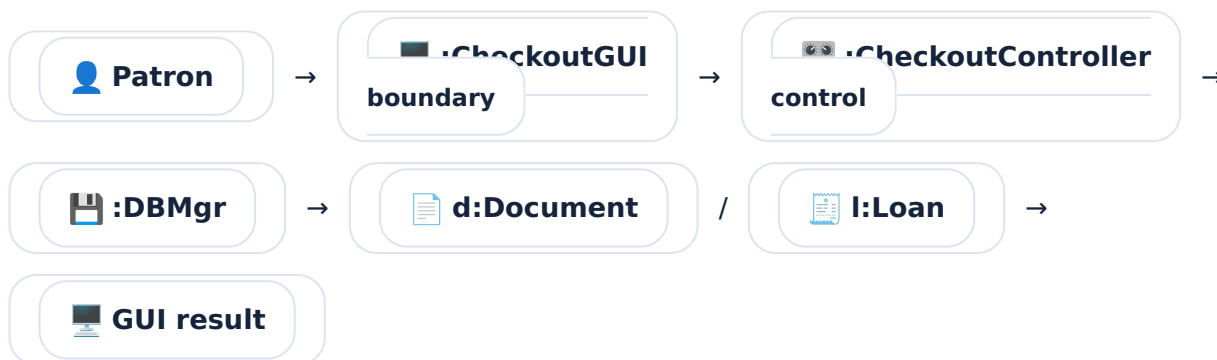
From Sequence Diagram to Implementation

Messages reveal responsibilities. Keep the boundary object focused on interaction and let the controller coordinate the use case.

TRACEABILITY REPAIR · LIBRARY CHECKOUT

Keep the same participants from analysis to implementation

Our earlier lessons established a clear collaboration. We keep it here rather than collapsing the GUI and controller into one object.



#	Sender	Message	Receiver	Responsibility exposed
1	Patron	checkout(uid, cnList)	CheckoutGUI	Capture the actor request.
2	CheckoutGUI	process(uid, cnList)	CheckoutController	Coordinate the use case.
3	Controller	getUser(uid)	DBMgr	Retrieve persistent user data.
4	Controller	getDocument(cn)	DBMgr	Retrieve each document.
5	Controller	isAvailable()	Document	Answer a domain-state question.
6	Controller	«create» Loan(u, d)	Loan	Create the checkout relationship.

#	Sender	Message	Receiver	Responsibility exposed
7	Controller	saveLoan(l)	DBMgr	Persist the new loan.
8	Controller	setAvailable(false)	Document	Change document state.
9	Controller	saveDocument(d)	DBMgr	Persist changed state.
10	Controller	msg	CheckoutGUI	Return the use-case result for presentation.

TRANSLATION RULES

What UML constructs become in code

Message → method call

The receiver normally owns the invoked operation.

loop → iteration

loop [for each cn] becomes a loop over cnList.

guard → condition

[u != null] and [available] become conditions.

create → constructor

«create» Loan(u,d) becomes new Loan(u,d).

state message → behavior

setAvailable(false) asks Document to change its own state.

persistence → DB responsibility

Save/retrieve messages become persistence operations.

IMPLEMENTATION

One coherent Java realization

Boundary: CheckoutGUI

```
public class CheckoutGUI {
    private final CheckoutController
    controller;

    public CheckoutGUI(CheckoutController
    controller) {
        this.controller = controller;
    }

    public String checkout(String uid,
    String[] cnList) {
        // capture input, delegate
        background work,
        // then display/return the result
        return controller.process(uid,
    cnList);
    }
}
```

Control: CheckoutController

```
public class CheckoutController {
    private final DBMgr dbm;

    public CheckoutController(DBMgr dbm) {
        this.dbm = dbm;
    }

    public String process(String uid,
    String[] cnList) {
        User u = dbm.getUser(uid);
        if (u == null) return "Unknown
        user";

        for (String cn : cnList) {
            Document d =
            dbm.getDocument(cn);
            if (d != null &&
            d.isAvailable()) {
                Loan l = new Loan(u, d);
                dbm.saveLoan(l);
                d.setAvailable(false);
                dbm.saveDocument(d);
            }
        }
        return "Checkout completed";
    }
}
```

Persistence collaborator: DBMgr

```
public class DBMgr {
    public User getUser(String uid) { /* retrieve */ }
    public Document getDocument(String cn) { /* retrieve */ }
    public void saveLoan(Loan loan) { /* persist */ }
    public void saveDocument(Document d) { /* persist */ }
}
```

Architecture lesson: the sequence diagram does not force one Java architecture, but it should remain internally coherent. Here the GUI is a boundary, the controller coordinates

the use case, domain objects own domain behavior, and DBMgr owns persistence operations.

Receiver → responsibility → method

Sequence message	Receiver	Likely implementation evidence
process(uid, cnList)	CheckoutController	controller.process(uid, cnList)
getUser(uid)	DBMgr	dbm.getUser(uid)
getDocument(cn)	DBMgr	dbm.getDocument(cn)
isAvailable()	Document	d.isAvailable()
«create» Loan(u,d)	Loan	new Loan(u,d)
setAvailable(false)	Document	d.setAvailable(false)

EXAM LENS

Do not copy arrows mechanically. Ask: WHO RECEIVES THIS MESSAGE? WHAT RESPONSIBILITY DOES THAT IMPLY? DOES THE IMPLEMENTATION PRESERVE THE SAME COLLABORATION STORY?

TOPIC 08 · LESSON 12 · 12 OF 14 LESSONS

LESSON 12

Worked Examples: Read Real UML Sequence Diagrams

Read real diagrams from top to bottom and explain the collaboration before worrying about implementation detail.

External study gallery · uml-diagrams.org

The first two are the core study examples. The final two are enrichment examples for students ready to read richer interactions.

CORE EXAMPLES — study these carefully

Focus on ordering, lifelines, messages, activations and common control frames.

External source visual referenced by the course page

1 · Online Bookshop Read it in this order: Web Customer → search inventory → search results → optional view description → optional add to cart → referenced Checkout interaction. Vertical position communicates ordering; loop/opt/ref frames communicate control structure. Open source example ↗

Source: <https://www.uml-diagrams.org/online-shopping-uml-sequence-diagram-example.html?context=seq-examples>

External source visual referenced by the course page

2 · Submit Comments to Pluck Read it in this order: user submits comment → website validates → DWR/AJAX transfers data → validation errors can return through callbacks → valid comment goes to Pluck → recent comments return. Study synchronous/asynchronous arrows, callbacks and execution bars. Open source example ↗

Source: <https://www.uml-diagrams.org/pluck-comments-uml-sequence-diagram-example.html>

EXPLORE MORE — enrichment, not required framework knowledge

Do not study Facebook, Spring, Hibernate or OAuth as technologies here. Use these only to practice reading more complex UML interactions.

External source visual referenced by the course page

3 · Facebook User Authentication UML reading goal: follow the protected-resource request through authorization and identify the alternative outcome when permission is denied. Open source example ↗

Source: <https://www.uml-diagrams.org/facebook-authentication-uml-sequence-diagram-example.html>

External source visual referenced by the course page

4 · Spring + Hibernate Transaction Management UML reading goal: trace nested calls and conditional commit/rollback behavior. The frameworks themselves are not learning outcomes for CPCS 351. Open source example ↗

Source: <https://www.uml-diagrams.org/examples/spring-hibernate-transaction-sequence-diagram-example.html>

Reference gallery: [UML Sequence Diagrams Examples ↗](#).

Transfer the reading skill to our course examples

**Fantasy Quest Reservation**

Player → QuestUI → ReservationController → QuestInventory → Party → Reservation → confirmation.

**Course Registration**

Student → RegistrationUI → Controller → Student/Section checks → Enrollment → result.

**Online Shopping**

Customer → Storefront → Cart → Inventory → Order/Payment → confirmation.



Clinic Appointment

Patient → AppointmentUI → Scheduler → ProviderAvailability → Appointment → confirmation.

Fully reasoned mini-example: Fantasy Quest

Step	Subject	Action	Object acted upon	Data
1	Player	requests reservation	QuestUI	questId, partyId
2	QuestUI	delegates reserve	ReservationController	questId, partyId
3	Controller	checks capacity	QuestInventory	questId
4	Controller	checks eligibility	Party	partyId
5	Controller	creates	Reservation	quest, party
6	Controller	returns confirmation	QuestUI	reservationId

TRANSFER SKILL

For every diagram: identify participants first, read downward, classify each message, locate branches/loops/creation/destruction, then explain the complete scenario in plain language.

TOPIC 08 · LESSON 13 · 13 OF 14 LESSONS

LESSON 13

Practice Studio

Practice the reasoning pipeline, not just UML notation.

**Challenge A · Registration**

Student requests a section. Identify nontrivial steps; write normal scenario/table; choose participants; add prerequisite, capacity and conflict guards; create Enrollment.

**Challenge B · Smart Parking**

Driver requests a space, pays, and receives reservation. Separate foreground from background before drawing.

**Challenge C · Repair causality**

A diagram calls `getUser(uid)` before the actor submits `uid`. Explain why the semantics are wrong and repair the order.

Submission recipe

1. Underline the actor trigger.
2. Mark trivial vs nontrivial use-case steps.
3. Write the normal scenario.
4. Convert at least six actions into a five-column scenario table.
5. Draw participants and messages in causal order.
6. Add guards/loop/create only where justified.
7. Explain two receiver responsibilities implied by messages.
8. Review the diagram for the mistakes in Lesson 14.

Check your reasoning — Challenge A

Attempt the registration challenge first. Then reveal one reasonable solution and compare the responsibility decisions, not merely the object names.

Reveal suggested solution

Possible participant chain



Possible normal interaction

1. Student submits the requested section to **RegistrationUI**.
2. UI delegates `register(studentId, sectionId)` to **RegistrationController**.
3. Controller retrieves the Student and Section.
4. Controller asks whether prerequisites are satisfied.
5. Controller checks section capacity.
6. Controller checks timetable conflicts.
7. If all conditions hold, Controller creates **Enrollment(student, section)**.
8. The enrollment is saved and a success result returns to the UI.

Typical guards: `[prerequisites satisfied]` `[capacity available]`

`[no timetable conflict]`

Responsibilities exposed by messages: the controller coordinates the use case; Student/Section answer domain questions; the Enrollment object represents the successful registration relationship. Other sound designs are possible if responsibilities remain coherent.

Self-test

If you can explain why every arrow exists and why it occurs at that exact point in time, you understand the model. If you only recognize the symbols, you do not yet.

TOPIC 08 · LESSON 14 · 14 OF 14 LESSONS

LESSON 14 · FINAL REVIEW

Common Mistakes, Agile Guidance & Exam Checklist

A sequence diagram can look neat and still be semantically wrong.

Common mistakes from the lecture

X Merge separate calls into one activation

Four calls to DBMgr are four executions, not one continuous execution interval.

X Incorrect nesting

If calls happen during `process(...)`, their execution must be nested within that control flow.

X Process before the trigger

`getUser(uid)` cannot happen before the Patron submits `uid`.

X Turn the diagram into code

OIM is high-level interaction design; omit unnecessary programming detail.

Agile principles

✓ **Work closely with customers/users** to understand business transactions.

✓ **Write scenarios and scenario tables collaboratively.**

✓ **Skip an artifact when the team already understands the scenario well.**

✓ **Convert scenario/table to a sequence diagram off-line when visualization adds value.**

✓ **Keep it simple; prototype when needed to remove doubt.**

Study-source coverage check

You should be able to...	Ready?
Explain foreground vs background processing	☐
Identify trivial/nontrivial use-case steps	☐
Write scenario sentences and a five-column table	☐
Translate the four subject/object cases to sequence messages	☐
Distinguish analysis from design sequence diagrams	☐
Read call parameters, types, returns and causal order	☐
Use loops, guards, creation and instance notation	☐
Map messages to responsibilities/methods	☐
Detect common semantic mistakes	☐

TOPIC 08 IN ONE LINE

**FIND BACKGROUND WORK → DESCRIBE COLLABORATION → STRUCTURE IT →
VISUALIZE TIME → ASSIGN RESPONSIBILITIES → INSPECT SEMANTICS.**